

UNIVERSIDAD MAYOR DE SAN ANDRES
CARRERA DE INFORMATICA



Informe actividades individuales

Estudiante: Ruben Ignacio Bautista Apaza

Ci: 12765200

RU: 1855760

LA PAZ - BOLIVIA

2026

a) Clasificación de texturas

“Desarrollar una aplicación capaz de diferenciar superficies dentro de una imagen, por ejemplo: césped, tierra, cemento o asfalto, aplicando principios similares a los utilizados en clasificación de imágenes satelitales.”

Para este ejercicio se uso un Windows Forms desde visual studio, en el cual se uso los siguientes componentes principales: 2 PictureBox, 2 buttons, 1 OpenFileDialog para poder abrir una imagen de mi computadora, y 1 label para mostrar mensajes.

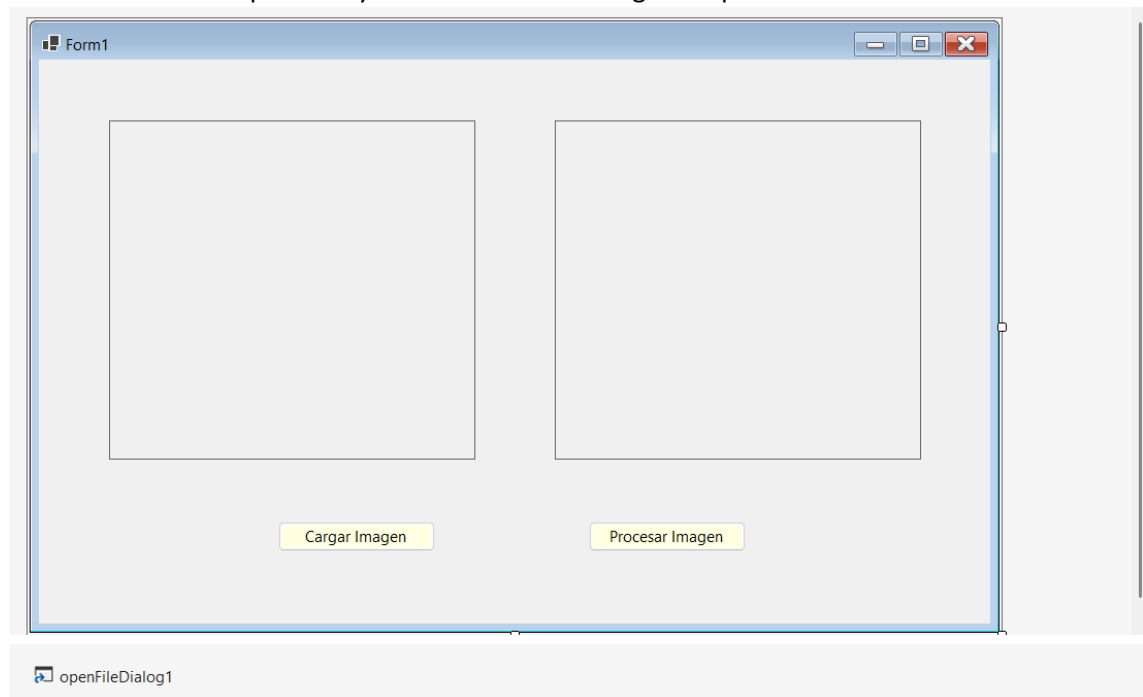
Para la parte del código, agregue un atributo de tipo Bitmap para reservar la imagen original

Dentro de la función para el primer botón, hice que a través del OpenFileDialog se pueda cargar una imagen y esta misma guardarlo en el atributo que ya había definido, luego asignaría la imagen cargada al pictureBox, y por ultimo mostraria un mensaje a traves del label1.

Para la función del botón 2, primero realizo la verificación que el atributo imagenOriginal no este nulo, esto verificando que se cargo una imagen.

En caso de que no sea nulo, lo que se hará es definir una nueva variable llamada “resultado”, la cual tendrá las mismas características de la imagen original y sobre la cual se iran realizando los cambios.

Dentro del botón 2 se llama a la función ClasificarPixel el cual lograra realizar la diferenciación de superficies y devolverá un color según lo que se detecte.



```
namespace a__Clasificacion_de_Texturas
{
```

```
    public partial class Form1 : Form
    {
        Bitmap imagenOriginal;

        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

1 referencia

```
private void button1_Click(object sender, EventArgs e)
{
    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        imagenOriginal = new Bitmap(openFileDialog1.FileName);

        pictureBox1.Image = imagenOriginal;

        label1.Text = "Imagen cargada correctamente";
    }
}
```

1 referencia

```
private void button2_Click(object sender, EventArgs e)
{
    if (imagenOriginal == null)
    {
        MessageBox.Show("Primero cargue una imagen.");
        return;
    }

    Bitmap resultado = new Bitmap(
        imagenOriginal.Width,
        imagenOriginal.Height);

    for (int y = 0; y < imagenOriginal.Height; y++)
    {
        for (int x = 0; x < imagenOriginal.Width; x++)
        {
            Color pixel = imagenOriginal.GetPixel(x, y);

            Color nuevoColor = ClasificarPixel(pixel);

            resultado.SetPixel(x, y, nuevoColor);
        }
    }

    pictureBox2.Image = resultado;

    label1.Text = "Clasificación finalizada";
}
```

1 referencia

```
private Color ClasificarPixel(Color c)
{
    int r = c.R;
    int g = c.G;
    int b = c.B;

    int brillo = (r + g + b) / 3;

    int rg = Math.Abs(r - g);
    int rb = Math.Abs(r - b);
    int gb = Math.Abs(g - b);

    if (g > r * 1.12 && g > b * 1.12)
    {
        return Color.LimeGreen;
    }

    if (brillo < 110 && rg < 35 && rb < 35 && gb < 35)
    {
        return Color.DimGray;
    }

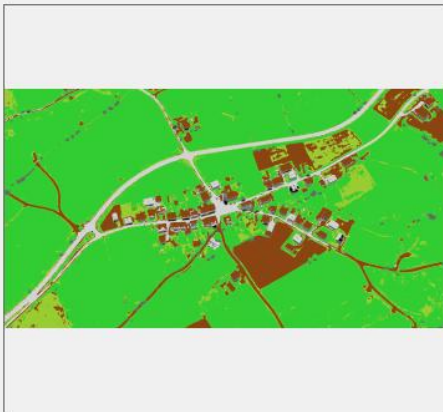
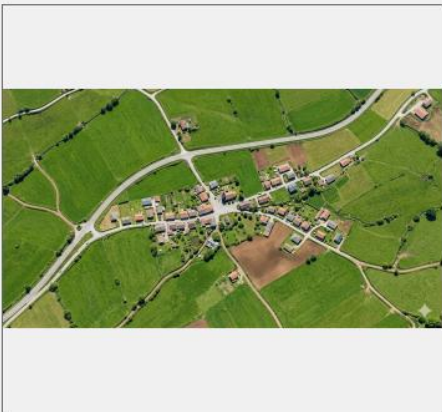
    if (brillo >= 110 && rg < 25 && rb < 25 && gb < 25)
    {
        return Color.LightGray;
    }

    if (r > g && g > b && r > 70)
    {
        return Color.SaddleBrown;
    }

    if (g > b && r > b && Math.Abs(r - g) < 40)
    {
        return Color.YellowGreen;
    }

    return Color.Black;
}
```

Form1



Clasificación finalizada

Cargar Imagen

Procesar Imagen

b) Implementación de un Filtro de Suavizado

Diseñar un filtro de promedio que recorra la imagen utilizando ventanas de 3x3 píxeles para reducir ruido y suavizar transiciones de color.

Para este ejercicio se utilizó un Windows Forms desde Visual Studio, en el cual se emplearon los siguientes componentes principales: 2 PictureBox, 2 Buttons, 1 OpenFileDialog para seleccionar una imagen desde la computadora y 1 Label para mostrar mensajes al usuario.

Para la parte del código, se definió un atributo de tipo Bitmap llamado imagenOriginal, cuya función es almacenar la imagen cargada para posteriormente aplicarle el filtro de suavizado.

Dentro de la función asociada al primer botón, se utilizó el OpenFileDialog para permitir al usuario seleccionar una imagen. Una vez seleccionada, esta se almacena en el atributo imagenOriginal y se muestra en el primer PictureBox. Finalmente, se actualiza el Label indicando que la imagen fue cargada correctamente.

Para la función del segundo botón, primero se verifica que el atributo imagenOriginal no sea nulo, asegurando que exista una imagen cargada antes de aplicar cualquier procesamiento. En caso de no existir una imagen, se muestra un mensaje de advertencia al usuario.

Posteriormente, se llama al método AplicarFiltroPromedio, el cual se encarga de generar una nueva imagen llamada resultado con las mismas dimensiones que la imagen original. Para cada píxel de la imagen, se analiza una ventana de 3x3 píxeles vecinos, sumando por separado los valores de los canales rojo, verde y azul.

Una vez obtenidas las sumas de los nueve píxeles de la ventana, se calcula el promedio de cada componente de color dividiendo entre nueve. Con estos nuevos valores se crea un nuevo color que reemplaza al píxel correspondiente en la imagen resultado.

Finalmente, para incrementar el efecto de suavizado, el filtro promedio se aplica repetidamente cinco veces sobre la imagen obtenida. Esto permite reducir variaciones bruscas entre píxeles y generar una imagen más uniforme y menos afectada por ruido. El resultado final se muestra en el segundo PictureBox

```
namespace b__Implementación_de_un_Filtro_de_Suavizado
{
    3 referencias
    public partial class Form1 : Form
    {
        Bitmap imagenOriginal;
        1 referencia
        public Form1()
        {
            InitializeComponent();
        }

        1 referencia
        private void button1_Click(object sender, EventArgs e)
        {
            if (openFileDialog1.ShowDialog() == DialogResult.OK)
            {
                imagenOriginal = new Bitmap(openFileDialog1.FileName);

                pictureBox1.Image = imagenOriginal;

                label1.Text = "Imagen cargada correctamente";
            }
        }
    }
}
```

2 referencias

```
private Bitmap AplicarFiltroPromedio(Bitmap imagen)
{
    Bitmap resultado = new Bitmap(
        imagen.Width,
        imagen.Height);

    for (int y = 1; y < imagen.Height - 1; y++)
    {
        for (int x = 1; x < imagen.Width - 1; x++)
        {
            int sumaR = 0;
            int sumaG = 0;
            int sumaB = 0;

            // Ventana 3x3
            for (int j = -1; j <= 1; j++)
            {
                for (int i = -1; i <= 1; i++)
                {
                    Color pixel = imagen.GetPixel(x + i, y + j);

                    sumaR += pixel.R;
                    sumaG += pixel.G;
                    sumaB += pixel.B;
                }
            }

            int promedioR = sumaR / 9;
            int promedioG = sumaG / 9;
            int promedioB = sumaB / 9;

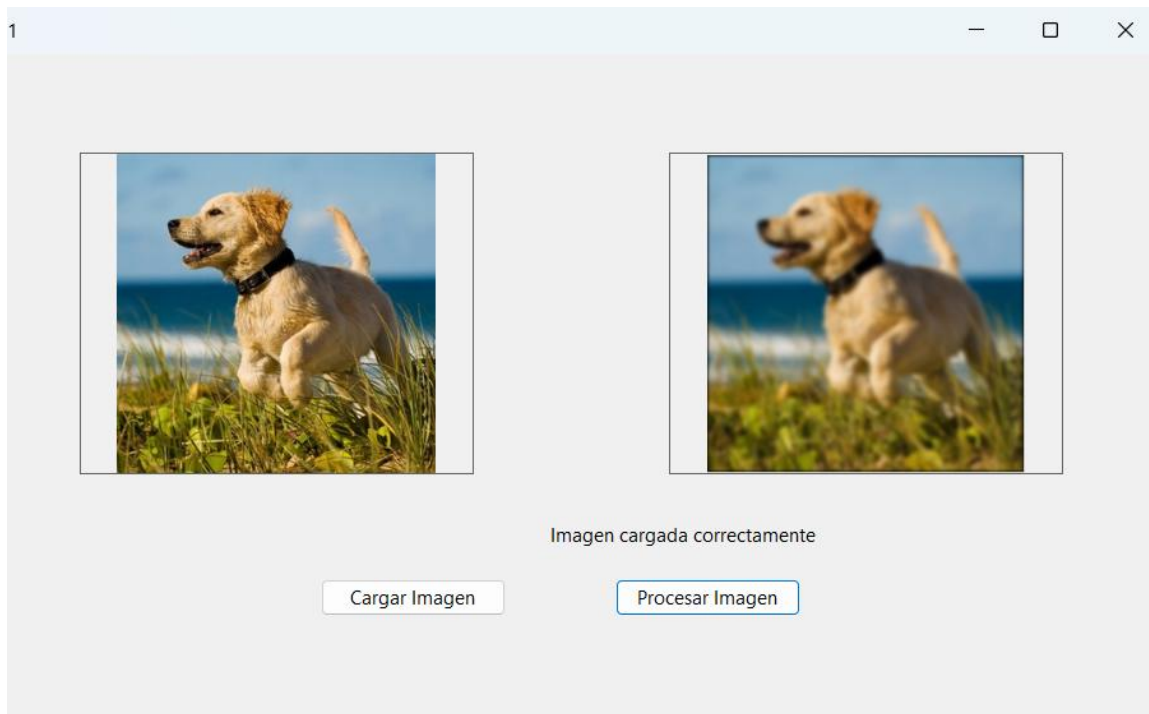
            resultado.SetPixel(
                x,
                y,
                Color.FromArgb(
                    promedioR,
                    promedioG,
                    promedioB));
        }
    }

    return resultado;
}
```

1 referencia

```
private void button2_Click(object sender, EventArgs e)
{
    if (imagenOriginal == null)
    {
        MessageBox.Show("Primero cargue una imagen.");
        return;
    }

    Bitmap resultado = AplicarFiltroPromedio(imagenOriginal);
    for (int i = 0; i < 5; i++)
    {
        resultado = AplicarFiltroPromedio(resultado);
    }
    pictureBox2.Image = resultado;
}
```



c) Cover de "La Vaca Lola"

Realizar una producción multimedia de la canción infantil utilizando la identidad de un artista al menos conocido, Vincular el audio con una animación (puede ser el modelo 3D o los monigotes) asegurando que el ritmo y el movimiento coincidan.

Para este ejercicio se utilizó Google Colab para aprovechar los recursos de procesamiento y ejecutar distintos modelos de inteligencia artificial obtenidos desde Hugging Face.

Inicialmente se instalaron las dependencias necesarias para el funcionamiento de los modelos, entre ellas **Qwen-TTS** para la generación y clonación de voz, **Flash-Attention** para optimizar el rendimiento y **Accelerate** para la administración de recursos de procesamiento.

Posteriormente se verificó si el entorno contaba con una GPU disponible. En caso de existir una GPU, el modelo utilizaría dicho recurso para acelerar el procesamiento; de lo contrario, se ejecutaría utilizando únicamente el procesador.

Para la clonación de voz se utilizó el modelo Qwen3-TTS obtenido desde Hugging Face. Una vez cargado el modelo, se proporcionó un archivo de audio, y para este caso del cantante BAD BUNNY como referencia, también añadiendo el texto de referencia, el cual indica las palabras exactas que se menciona en el archivo de audio. A partir de estos datos, el modelo analizó las características de la voz, tales como el tono, la entonación y el estilo de pronunciación, generando una estructura denominada `voice_clone_prompt` que posteriormente sería utilizada para sintetizar nuevas frases manteniendo características similares a la voz de referencia.

Luego se definió la letra que sería interpretada por la voz clonada. Mediante el método `generate_voice_clone`, el modelo generó un nuevo archivo de audio pronunciando el texto especificado y conservando las características aprendidas de la voz original. Finalmente, el resultado fue almacenado en formato WAV para su posterior utilización.

Instalar Dependencias

```
▶ # @title Instalar Dependencias
!pip install -U qwen-tts
!pip install -U flash-attn --no-build-isolation
!pip install -U accelerate

▶ import torch
import soundfile as sf
from IPython.display import Audio, display
import os

# Verificar si CUDA (GPU) está disponible
device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Usando dispositivo: {device}")

if device == "cuda":
    print(f"GPU: {torch.cuda.get_device_name(0)}")
else:
    print("Advertencia: Ejecutando en CPU. La generación será lenta.")

# Función auxiliar para reproducir audio directamente en el notebook
def play_audio(file_path):
    if os.path.exists(file_path):
        display(Audio(file_path))
    else:
        print(f"Archivo no encontrado: {file_path}")

▶ from qwen_tts import Qwen3TTSModel

# @title Seleccionar y Cargar Modelo Base
model_name = "Qwen/Qwen3-TTS-12Hz-1.7B-Base" # @param ["Qwen/Qwen3-TTS-12Hz-1.7B-Base", "Qwen/Qwen3-TTS-12Hz-0.6B-Base"]

print(f"Cargando {model_name}...")

# Cargar el modelo Base/Clone
base_model = Qwen3TTSModel.from_pretrained(
    model_name,
    device_map="auto",
    dtype=torch.bfloat16 if device == "cuda" else torch.float32,
)

print("¡Modelo de Clonación de Voz Cargado!")

▶ ref_audio_path = "badbunny.mp3"
ref_audio_text = "Debi tirar mas fotos de cuando te tuve debi tirar mas besos y al

print(f"Clonando voz desde referencia...")

# 1. Crear estructura del prompt (opcional pero recomendado para reusar)
# Esto extrae las características de estilo del audio de referencia
voice_clone_prompt = base_model.create_voice_clone_prompt(
    ref_audio=ref_audio_path,
    ref_text=ref_audio_text
)
```



```

# 2. Generar - Spanish example
target_text = "La vaquita baila suave bajo el sol tropical, moviendo la cola al ritmo reggae junto al palmar. E E, La vaquita,

wavs, sr = base_model.generate_voice_clone(
    text=target_text,
    language="Spanish",
    voice_clone_prompt=voice_clone_prompt
)

output_clone_path = "output_cloned4.wav"
sf.write(output_clone_path, wavs[0], sr)
|
print("Referencia Original:")
print(f"(Fuente de referencia: {ref_audio_path})")

print("\nResultado Clonado:")
play_audio(output_clone_path)

```

Una vez obtenida la voz, se procedió a generar la base musical utilizando el modelo MusicGen de Meta AI, también disponible en Hugging Face. Para ello se definió un prompt descriptivo indicando el estilo musical deseado (lo escribí en inglés para tener un mejor resultado), especificando características como ritmo salsa, ambiente tropical, instrumentos latinos y una atmósfera alegre orientada al público infantil (imitando un poco a las últimas canciones de Bad Bunny).

A partir de dicha descripción, el modelo generó automáticamente una pieza musical instrumental. El resultado se almacenó inicialmente en formato WAV y posteriormente fue convertido a formato MP3

```

import scipy.io.wavfile
from transformers import pipeline
from pydub import AudioSegment
from IPython.display import Audio, display

```

```

❶ # Cargar modelo
synthesiser = pipeline(
    "text-to-audio",
    "facebook/musicgen-small",
    device=0
)

prompt = """
Happy children's salsa song, tropical Caribbean atmosphere,
catchy melody, upbeat salsa rhythm, congas, bongos,
timbales, piano montuno, brass section, joyful and danceable,
family friendly, playful and cheerful mood, summer beach feeling,
memorable chorus, professional studio quality, clean mix,
vibrant Latin energy, suitable for children singing and dancing.
"""

# Generar música
music = synthesiser(
    prompt,
    forward_params={
        "do_sample": True,
        "max_new_tokens": 1500
    }
)

# Guardar WAV
wav_file = "musica_salsa.wav"

scipy.io.wavfile.write(
    wav_file,
    rate=music["sampling_rate"],
    data=music["audio"]
)

# Convertir a MP3
audio = AudioSegment.from_wav(wav_file)
mp3_file = "musica_salsa.mp3"
audio.export(mp3_file, format="mp3")

print("Archivo WAV:", wav_file)
print("Archivo MP3:", mp3_file)

# Mostrar reproductor
display(Audio(wav_file))

```

Posteriormente se realizó el proceso de mezcla entre la voz generada y la música instrumental utilizando la librería Pydub. Primero se cargaron ambos archivos de audio. Luego se incrementó la duración de la voz mediante repeticiones para obtener una presencia más prolongada dentro de la composición final (ya que la voz generada es mas corta que el tiempo de la cancion) tambien se redujo el volumen de la música de fondo para evitar que esta ocultara la voz principal.

A continuación, se verificó la duración de ambos audios. Si la música era más corta que la voz, esta era repetida automáticamente hasta alcanzar una duración suficiente. Posteriormente se recortaba al tamaño exacto de la pista vocal para asegurar una sincronización adecuada.

Finalmente, se realizó la superposición de ambas pistas, obteniendo una composición que combina la música generada por inteligencia artificial con la voz sintetizada a partir de la referencia proporcionada. El resultado fue exportado como un archivo MP3 denominado resultado.mp3, constituyendo el cover generado para la canción infantil.

```

from pydub import AudioSegment
from IPython.display import Audio, display

```

```

# Cargar audios
musica = AudioSegment.from_file("musica_salsa.mp3")
voz = AudioSegment.from_file("output_cloned4.wav")

voz = voz * 3

# Bajar volumen de la música
musica = musica - 12

# Ajustar la música al tamaño de la voz
if len(musica) < len(voz):
    while len(musica) < len(voz):
        musica += musica

musica = musica[:len(voz)]

# Superponer
final = musica.overlay(voz)

# Guardar
final.export("resultado.mp3", format="mp3")

print("Audio generado: resultado.mp3")

# Reproducir
display(Audio("resultado.mp3"))

```

Con este resultado de la mezcla de generación de música y clonación de voz, lo que realice fue añadirlo a una escena 3D en unity.

Antes de mostrar como se puso la música en el proyecto indicare como se armo la escena en unity.

Lo primero que hice fue descargar una animación desde MIXAMO, la cual subi a mi proyecto de unity, luego de eso lo que hice fue añadir un "plane", el cual representaría el suelo de la escena.

Posterior a eso también cree un Animator Controller para poder añadirlo a mi personaje y desde el cual agregaría la animación del personaje desde el animator

Luego de esto, añadi la música generada (se hizo en los primeros pasos) a los assets de mi proyecto y luego de esto añadi un nuevo objeto a mi proyecto el cual es Audio Source, en el cual desde el inspector añadi la canción del cover de la vaca lola, y marque la opcion de loop para que la musica no se acabe mientras se ejecuta la animación.

